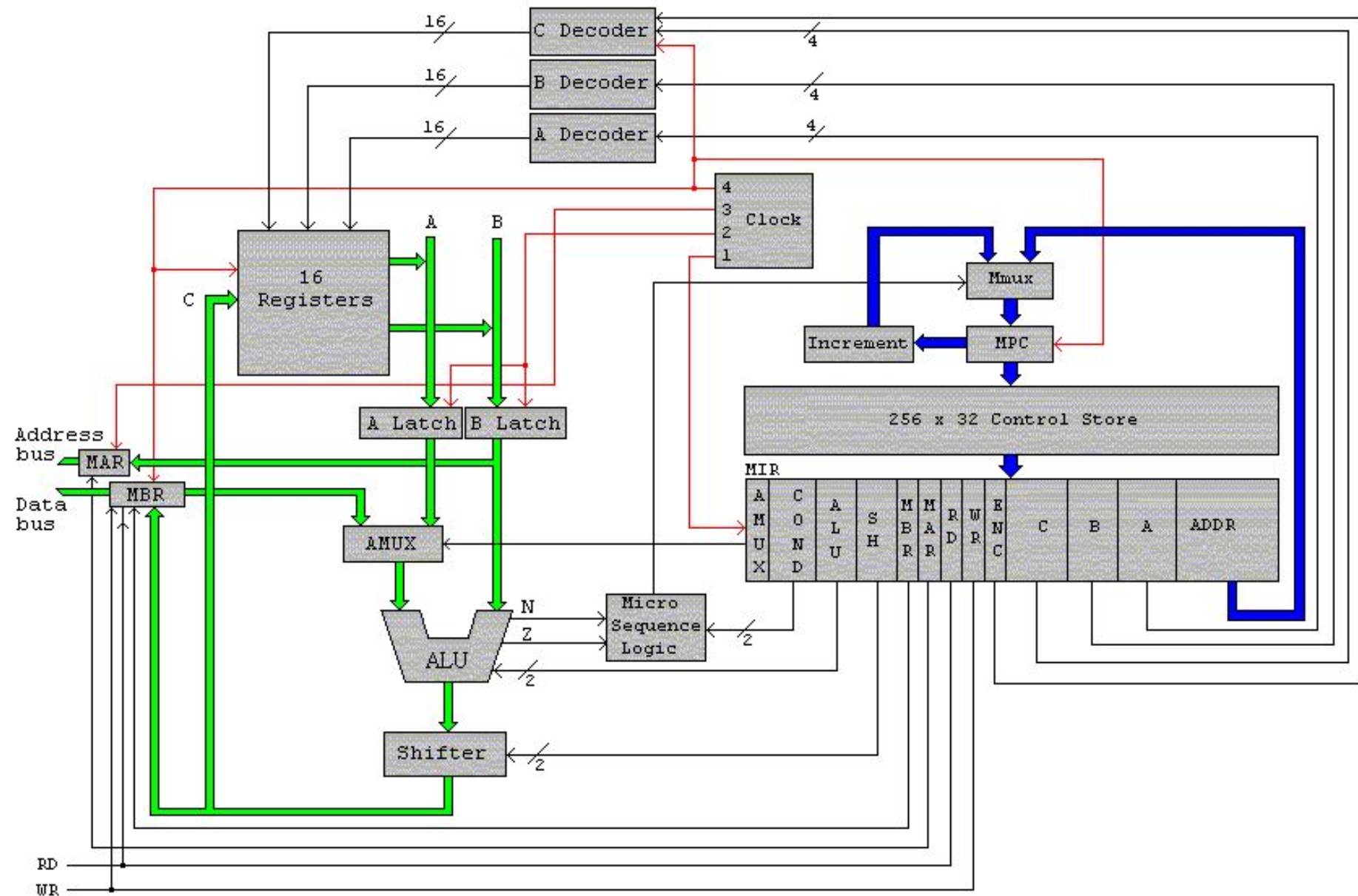


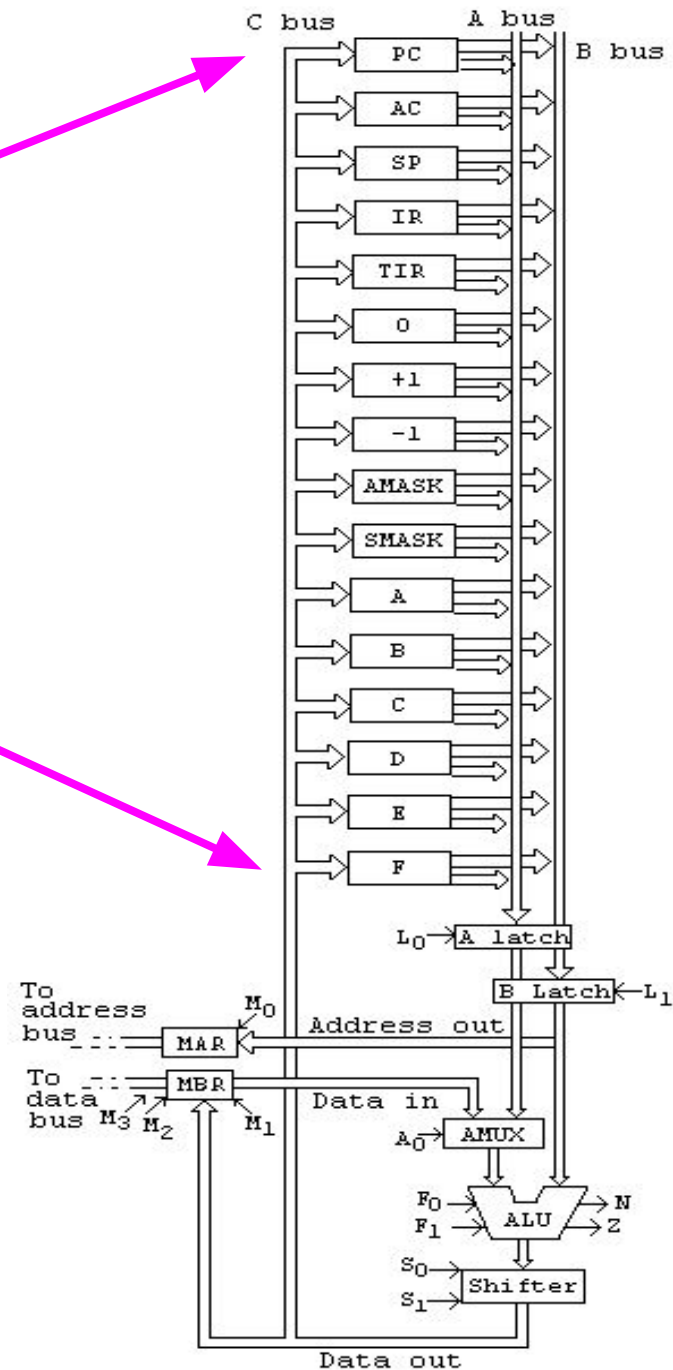
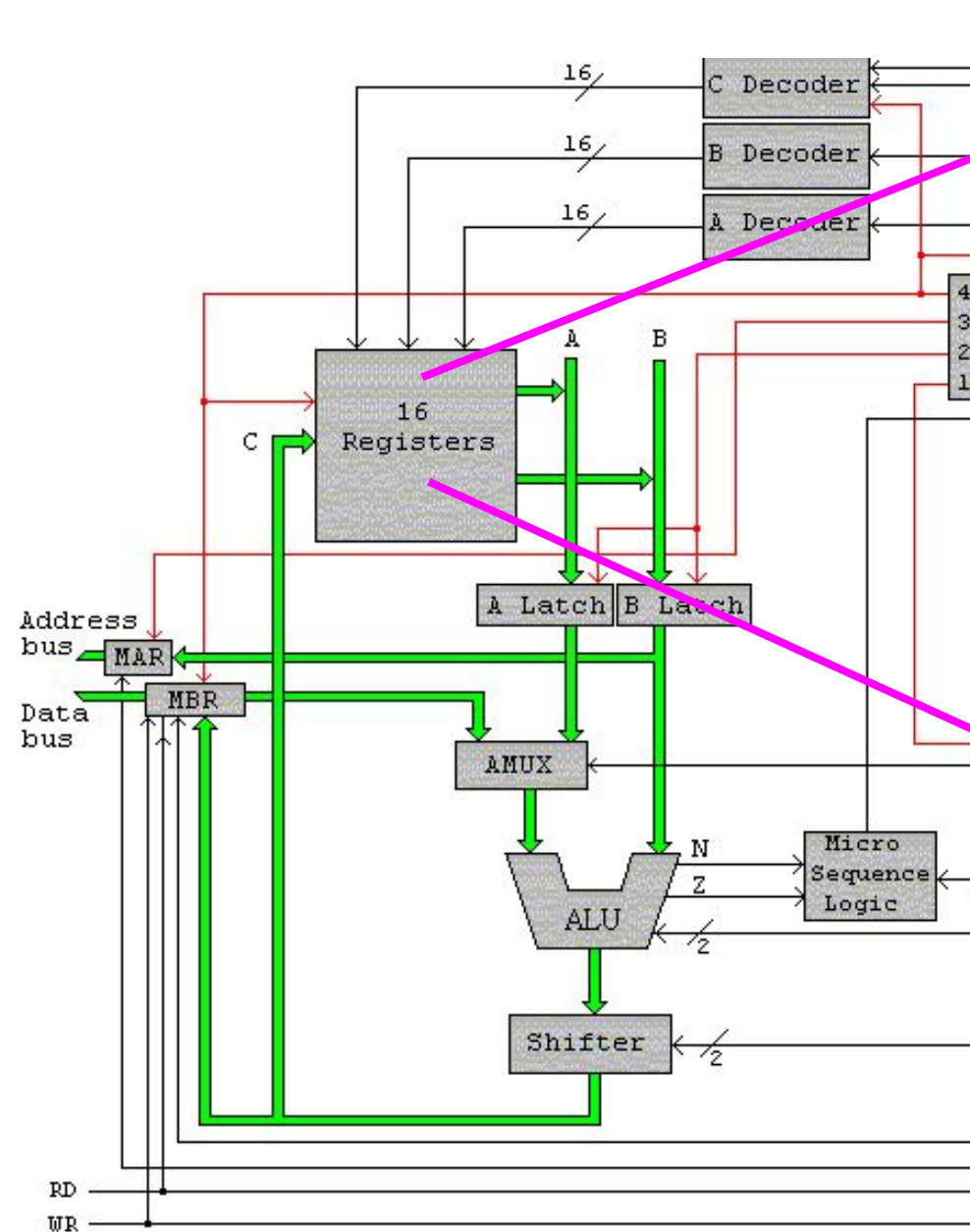
Arquitectura Horizontal

T2 - Ejecución del micro programa

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta





Binary	Mnemonic	Instruction	Meaning
0000xxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxx	\$TOD	Store direct	m[x]:=ac
0010xxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxx	\$UBD	Subtract direct	ac:=ac-m[x]
0100xxxxxx	JPOS	Jump positive	if ac->0 then pc:=x
0101xxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxx	JUMP	Jump	pc:=x
0111xxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxx	\$TOL	Store local	m[x+sp]:=ac
1010xxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxx	\$UBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	\$SHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	\$OPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	\$WAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<-y<=255)
11111110yyyyyyyy	DE\$P	Decrement sp	sp:=sp-y (0<-y<=255)

Binary	Mnemonic	Instruction	Meaning
0000xxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxx	JPOS	Jump positive	if ac->0 then pc:=x
0101xxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxx	JUMP	Jump	pc:=x
0111xxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<-y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<-y<=255)

STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

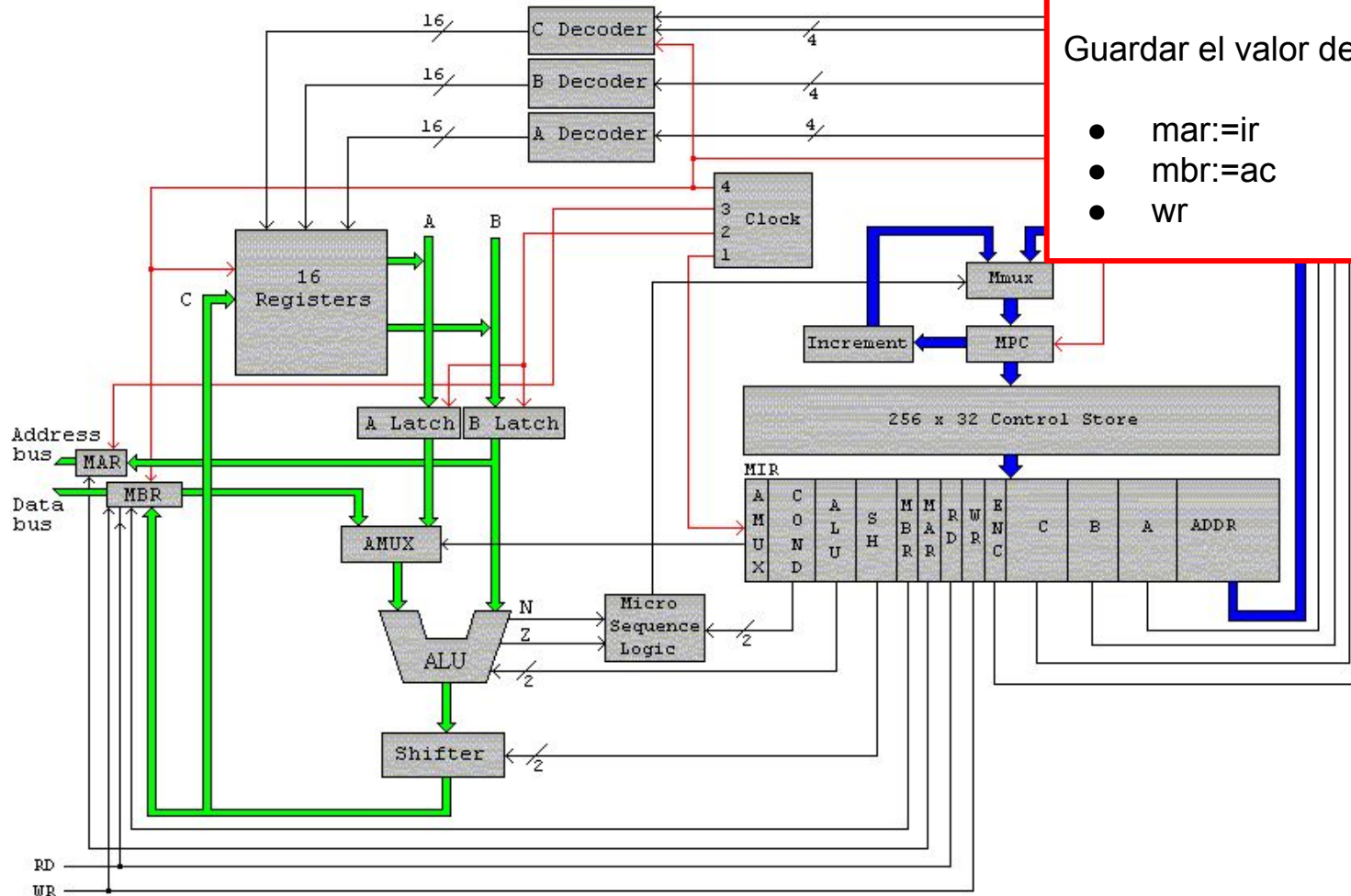
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]

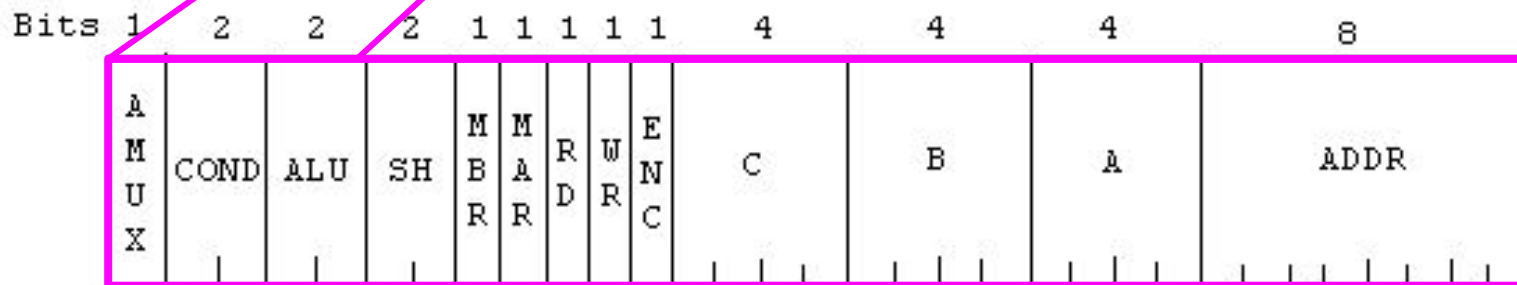
STORE DIRECT

...
STOD 9
...

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr





MBR, MAR, RW, WR, ENC

0 = No

1 = Yes

3 = (not used)

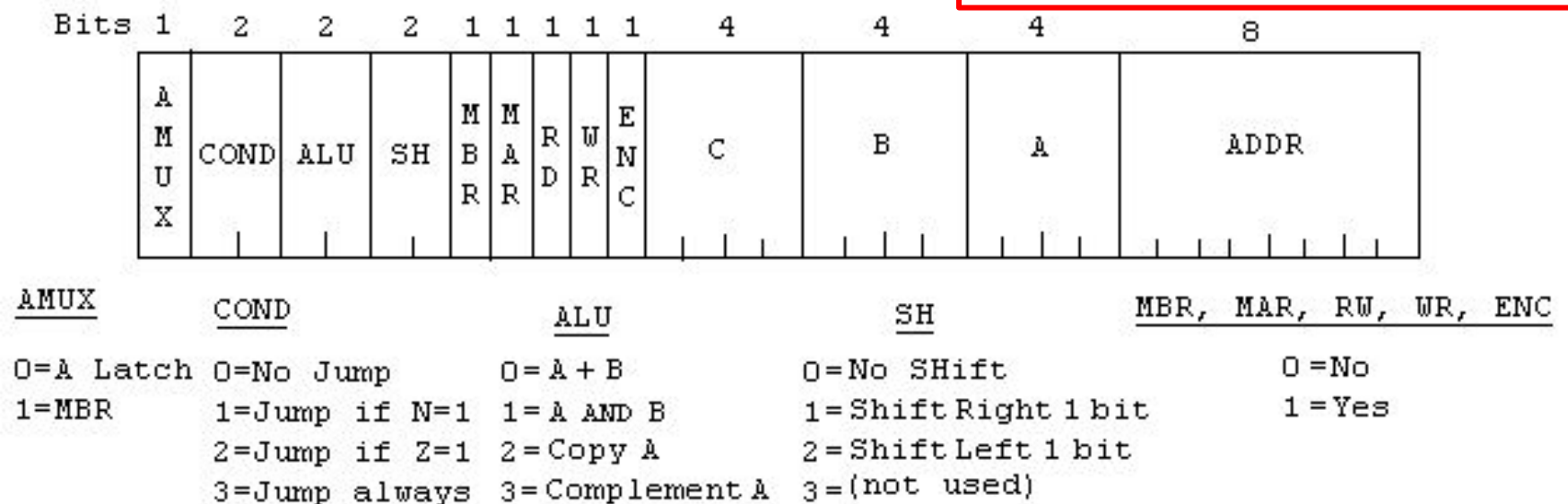
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	

STORE DIRECT

STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac

	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir; mbr := ac; wr;	0	0	2	0	1	1	0	1	0	0	3	1	00

STORE DIRECT

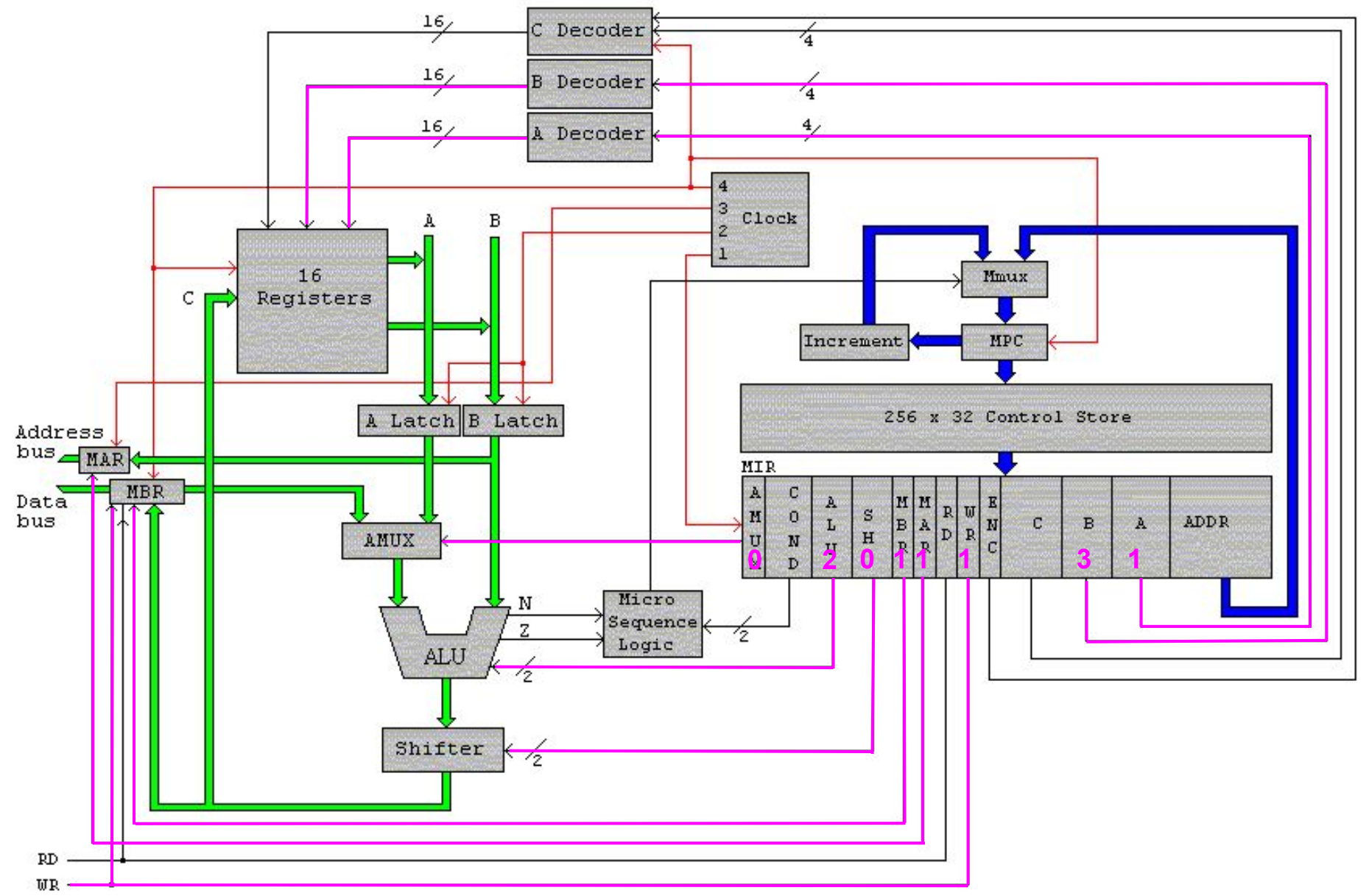
STOD 9

Guardar el valor del AC en la memoria(9)

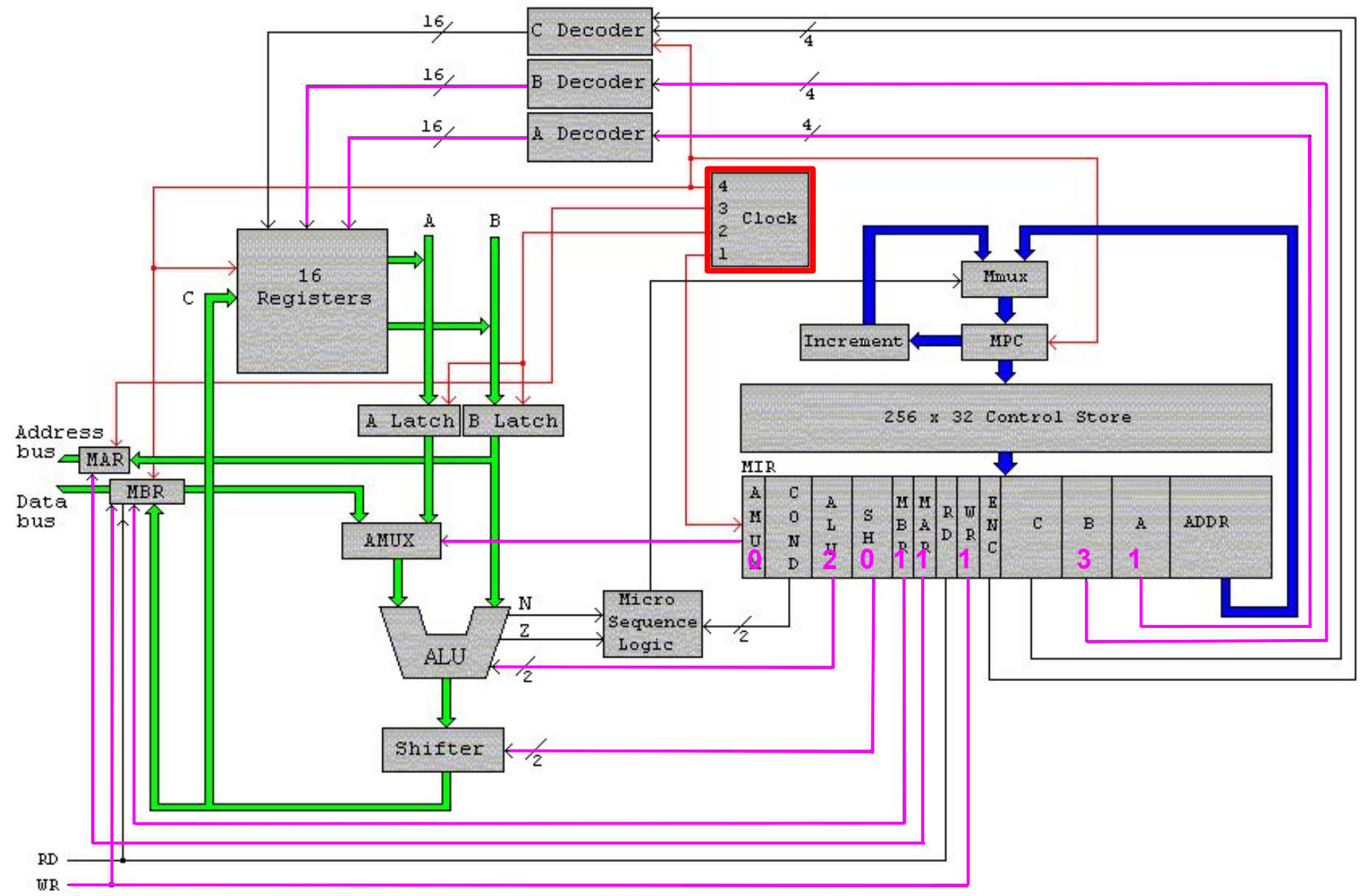
- mar:=ir
- mbr:=ac
- wr

Bits	1	2	2	2	1	1	1	1	1	4	4	4	8
	A M U X	COND	ALU	SH	M B R	M A R	R D	W R	E N C	C	B	A	ADDR
<u>AMUX</u>	<u>COND</u>		<u>ALU</u>		<u>SH</u>		<u>MBR, MAR, RD, WR, ENC</u>						
0=A Latch	0=No Jump		0=A + B		0=No Shift		0=No						
1=MBR	1=Jump if N=1		1=A AND B		1=Shift Right 1 bit		1=Yes						
	2=Jump if Z=1		2=Copy A		2=Shift Left 1 bit								
	3=Jump always		3=Complement A		3=(not used)								

0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir; mbr := ac; wr;				0	0	2	0	1	1	0	1	0	0	3	1	00



0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
mar := ir; mbr := ac; wr;				0	0	2	0	1	1	0	1	0	0	3	1	00



0: $mar := pc; rd;$	{ciclo principal}	40: $tir := lshift(tir);$ if n then goto 46;	{110x o 111x?}
1: $pc := pc + 1; rd;$	{incrementa pc}	41: $alu := tir;$ if n then goto 44;	{1100 o 1101?}
2: $ir := mbr;$ if n then goto 28;	{salva y decodifica mbr}	42: $alu := ac;$ if n then goto 22;	{1100 = JNEG}
3: $tir := lshift(ir + ir);$ if n then goto 19;		43: goto 0;	
4: $tir := lshift(tir);$ if n then goto 11;	{000x o 001x?}	44: $alu := ac;$ if z then goto 0;	{1101 = JNZE}
5: $alu := tir;$ if n then goto 9;	{0000 o 0001?}	45: $pc := band(ir, amask);$ goto 0;	
6: $mar := ir; rd;$	{0000 = LODD}	46: $tir := lshift(tir);$ if n then goto 50;	
7: $rd;$		47: $sp := sp + (-1);$	{1110 = CALL}
8: $ac := mbr;$ goto 0;		48: $mar := sp; mbr := pc; wr;$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}	49: $pc := band(ir, amask); wr;$ goto 0;	
10: $wr;$ goto 0;		50: $tir := lshift(tir);$ if n then goto 65;	{111, examina addr}
11: $alu := tir;$ if n then goto 15;	{0010 o 0011?}	51: $tir := lshift(tir);$ if n then goto 59;	
12: $mar := ir; rd;$	{0010 = ADDD}	52: $alu := tir;$ if n then goto 56;	
13: $rd;$		53: $mar := ac; rd;$	{1111000 = PSHI}
14: $ac := mbr + ac;$ goto 0;		54: $sp := sp + (-1); rd;$	
15: $mar := ir; rd;$	{0011 = SUBD}	55: $mar := sp; wr;$ goto 10;	
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 + no\ y$ }	56: $mar := sp; sp := sp + 1; rd;$	{1111001 = POPI}
17: $a := inv(mbr);$		57: $rd;$	
18: $ac := ac + a;$ goto 0;		58: $mar := ac; wr;$ goto 10;	
19: $tir := lshift(tir);$ if n then goto 25;	{010x o 011x?}	59: $ac := mbr; tir := lshift(tir);$ if n then goto 62;	
20: $alu := tir;$ if n then goto 23;	{0100 o 0101?}	60: $sp := sp + (-1);$	
21: $alu := ac;$ if n then goto 0;	{0100 = JZ}	61: $mar := sp; mbr := ac; wr;$ goto 10;	
22: $pc := band(ir, amask);$ goto 0;	{realiza el salto}	62: $mar := sp; sp := sp + 1; rd;$	{1111010 = PUSH}
23: $alu := ac;$ if z then goto 22;	{0101 = JZER}	63: $rd;$	
24: goto 0;	{no se produce el salto}	64: $ac := mbr;$ goto 0;	{1111011 = POP}
25: $alu := tir;$ if n then goto 27;	{0110 o 0111?}	65: $tir := lshift(tir);$ if n then goto 73;	
26: $pc := band(ir, amask);$ goto 0;	{0110 = JUMP}	66: $alu := tir;$ if n then goto 70;	
27: $ac := band(ir, amask);$ goto 0;	{0111 = LOCO}	67: $mar := sp; sp := sp + 1; rd;$	{1111100 = RETN}
28: $tir := lshift(ir + ir);$ if n then goto 40;	{10xx o 11xx?}	68: $rd;$	
29: $tir := lshift(tir);$ if n then goto 35;	{100x o 101x?}	69: $pc := mbr;$ goto 0;	
30: $alu := tir;$ if n then goto 33;	{1000 o 1001?}	70: $a := ac;$	{1111101 = SWAP}
31: $a := ir + sp;$	{1000 = LODL}	71: $ac := sp;$	
32: $mar := a; rd;$ goto 7;		72: $sp := a;$ goto 0;	
33: $a := ir + sp;$	{1001 = STOL}	73: $alu := tir;$ if n then goto 76;	
34: $mar := a; mbr := ac; wr;$ goto 10;		74: $a := band(ir, smask);$	{1111110 = INSP}
35: $alu := tir;$ if n then goto 38;	{1010 o 1011?}	75: $sp := sp + a;$ goto 0;	
36: $a := ir + sp;$	{1010 = ADDL}	76: $a := band(ir, smask);$	{1111111 = DESP}
37: $mar := a; rd;$ goto 13;		77: $a := inv(a);$	
38: $a := ir + sp;$	{1011 = SUBL}	78: $a := a + 1;$ goto 75;	
39: $mar := a; rd;$ goto 16;			

El Micro-Programa

Binary	Mnemonic	Instruction	Meaning
0000xxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxx	JPOS	Jump positive	if ac>0 then pc:=x
0101xxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxx	JUMP	Jump	pc:=x
0111xxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxx	ADDL	Add local	ac:= ac+m[sp+x]
1011xxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=-pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=-m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=-m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=-ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=-ac; ac:=-sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<-y<-255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=-sp-y (0<-y<-255)

```

40: tir := lshift(tir); if n then goto 46;      {110x 0 111x?}
41: alu := tir; if n then goto 44;              {1100 0 1101?}

42: alu := ac; if n then goto 22;              {1100 = JNEG}
43: goto 0;

44: alu := ac; if z then goto 0;                {1101 = JNZE}
45: pc := band(ir, amask); goto 0;

46: tir := lshift(tir); if n then goto 50;

47: sp := sp + (-1);                            {1110 = CALL}
48: mar := sp; mbr := pc; wr;
49: pc := band(ir, amask); wr; goto 0;

50: tir := lshift(tir); if n then goto 65;      {111, examina addr}
51: tir := lshift(tir); if n then goto 59;
52: alu := tir; if n then goto 56;

53: mar := ac; rd;                              {1111000 = PSHI}
54: sp := sp + (-1); rd;
55: mar := sp; wr; goto 10;

56: mar :=sp; sp := sp + 1; rd;                  {1111001 = POPI}
57: rd;
58: mar := ac; wr; goto 10;

59: alu := tir; if n then goto 62;

60: sp := sp + (-1);                            {1111010 = PUSH}
61: mar := sp; mbr := ac; wr; goto 10;

62: mar := sp; sp := sp + 1; rd;                  {1111011 = POP}
63: rd;
64: ac := mbr; goto 0;

65: tir := lshift(tir); if n then goto 73;
66: alu := tir; if n then goto 70;

67: mar := sp; sp := sp + 1; rd;                  {1111100 = RETN}
68: rd;
69: pc := mbr; goto 0;

70: a := ac;                                      {1111101 = SWAP}
71: ac := sp;
72: sp := a; goto 0;

73: alu := tir; if n then goto 76;

74: a := band(ir, smask);                        {1111110 = INSP}
75: sp := sp + a; goto 0;

76: a := band(ir, smask);                        {1111111 = DESP}
77: a := inv(a);
78: a := a + 1; goto 75;

```

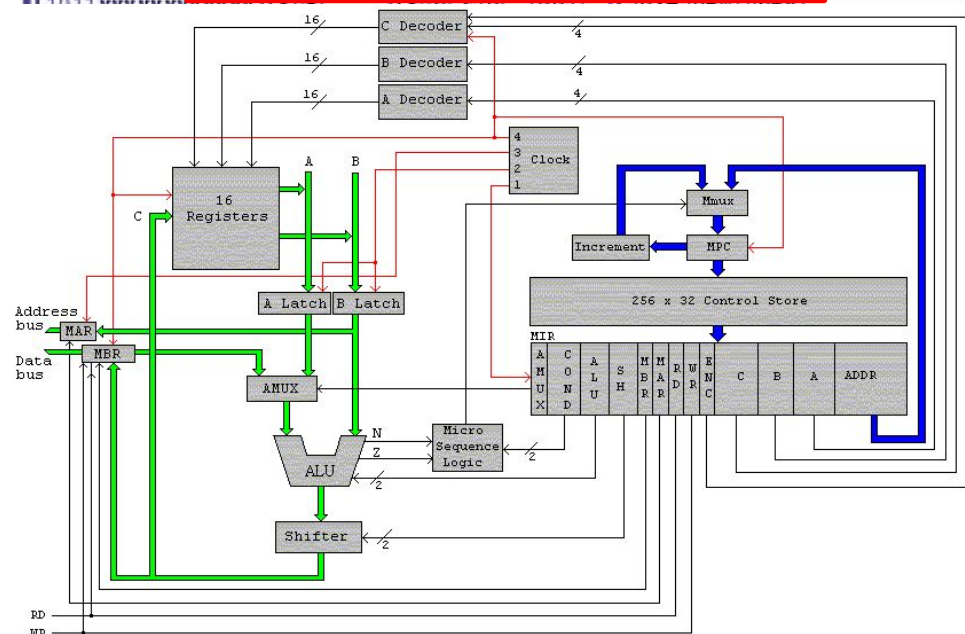
0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{000x o 001x?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 o 0001?}
6: $mar := ir; rd;$	{0000 = LODD}
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}
31: $a := ir + sp;$	{1000 = LODL}
32: $mar := a; rd; \text{goto 7;}$	
33: $a := ir + sp;$	{1001 = STOL}
34: $mar := a; mbr := ac; wr; \text{goto 10;}$	
35: $alu := tir; \text{if } n \text{ then goto 38;}$	{1010 o 1011?}
36: $a := ir + sp;$	{1010 = ADDL}
37: $mar := a; rd; \text{goto 13;}$	
38: $a := ir + sp;$	{1011 = SUBL}
39: $mar := a; rd; \text{goto 16;}$	

Binary	Mnemonic	Instruction	Meaning
0000xxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxx	ADDD	Add direct	$ac := ac + m[x]$
0011xxxxxx	SUBD	Subtract direct	$ac := ac - m[x]$
0100xxxxxx	JPOS	Jump positive	if $ac > 0$ then $pc := x$
0101xxxxxx	JZER	Jump zero	if $ac = 0$ then $pc := x$
0110xxxxxx	JUMP	Jump	$pc := x$
0111xxxxxx	LOCO	Load constant	$ac := x$ ($0 < x < 4095$)
1000xxxxxx	LODL	Load local	$ac := m[sp+x]$
1001xxxxxx	STOL	Store local	$m[x+sp] := ac$
1010xxxxxx	ADDL	Add local	$ac := ac + m[sp+x]$
1011xxxxxx	SUBL	Subtract local	$ac := ac - m[sp+x]$
1100xxxxxx	JNEG	Jump negative	if $ac < 0$ then $pc := x$
1101xxxxxx	JNZE	Jump nonzero	if $ac < > 0$ then $pc := x$
1110xxxxxx	CALL	Call procedure	$sp := sp - 1; m[sp] := pc; pc := x$
111100000000	PSHI	Push indirect	$sp := sp - 1; m[sp] := m[ac]$
11110010000000	POPI	Pop indirect	$m[ac] := m[sp]; sp := sp + 1$
11110100000000	PUSH	Push onto stack	$sp := sp - 1; m[sp] := ac$
11110110000000	POP	Pop from stack	$ac := m[sp]; sp := sp + 1$
11111000000000	RETN	Return	$pc := m[sp]; sp := sp + 1$
11111010000000	SWAP	Swap ac sp	$tmp := ac; ac := sp; sp := tmp$
11111100yyyyyy	INSP	Increment sp	$sp := sp + y$ ($0 < y < 255$)
11111110yyyyyy	DESP	Decrement sp	$sp := sp - y$ ($0 < y < 255$)

0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}
31: $a := ir + sp;$	{1000 = LODL}
32: $mar := a; rd; \text{goto 7;}$	
33: $a := ir + sp;$	{1001 = STOL}
34: $mar := a; mbr := ac; wr; \text{goto 10;}$	
35: $alu := tir; \text{if } n \text{ then goto 38;}$	{1010 o 1011?}
36: $a := ir + sp;$	{1010 = ADDL}
37: $mar := a; rd; \text{goto 13;}$	
38: $a := ir + sp;$	{1011 = SUBL}
39: $mar := a; rd; \text{goto 16;}$	

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

3F7 ...
 3F8 STOD 9
 3F9 JUMP 3F7
 3FA ...



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxx			
0011xxxxxx			
0100xxxxxx			
0101xxxxxx			
0110xxxxxx			
0111xxxxxx			
1000xxxxxx			
1001xxxxxx			
1010xxxxxx			
1011xxxxxx			

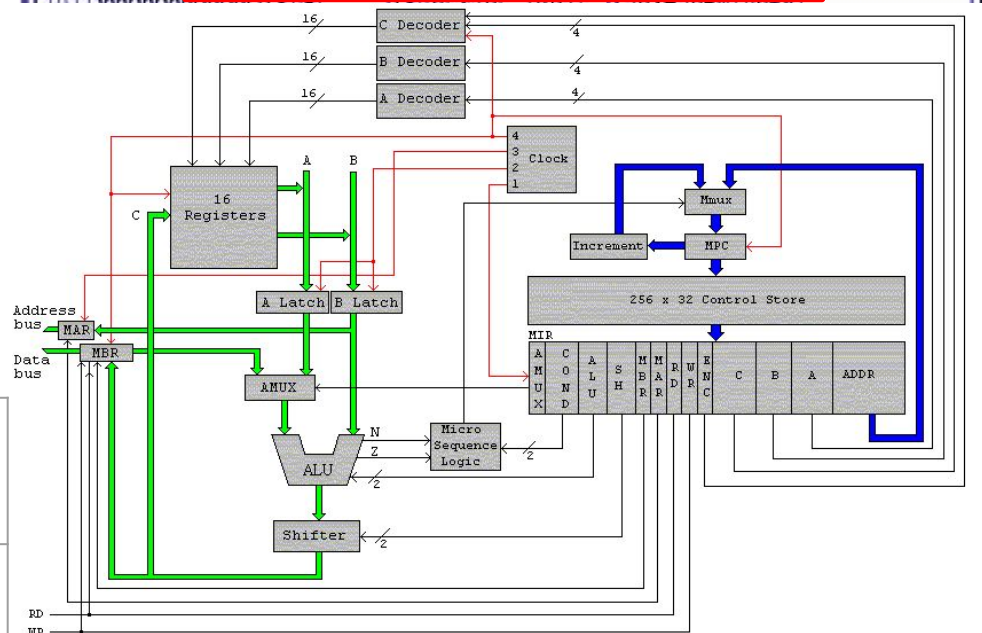
3F7 ...

3F8 STOD 9

3F9 JUMP 3F7

3FA ...

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
		3F8	0041						



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

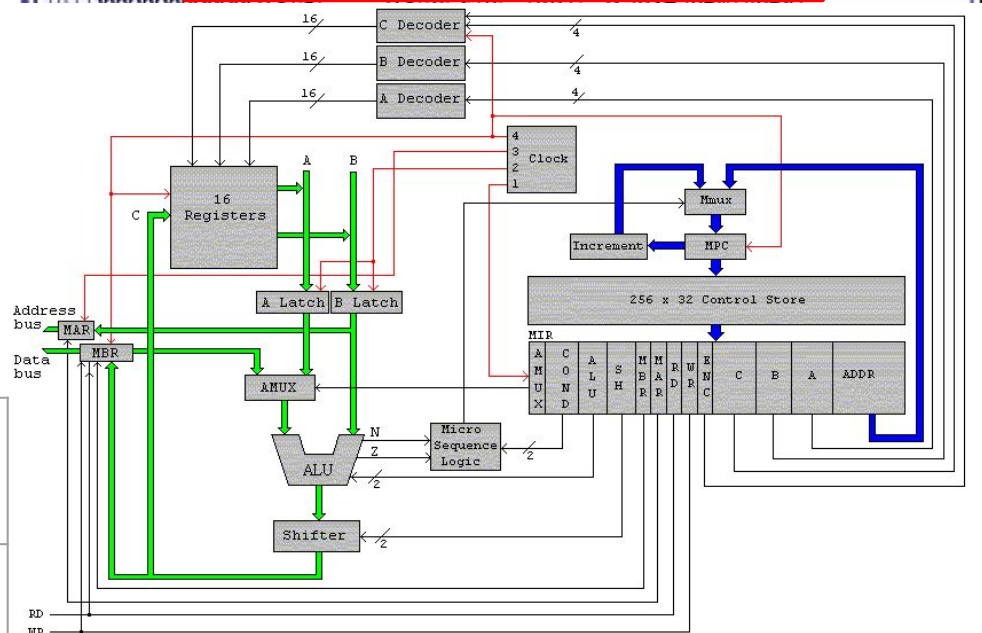
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
		3F8	0041						



```

0: mar := pc; rd;
1: pc := pc + 1; rd;
2: ir := mbr; if n then goto 28;
3: tir := lshift(ir + ir); if n then goto 19;
4: tir := lshift(tir); if n then goto 11;
5: alu := tir; if n then goto 9;

6: mar := ir; rd;
7: rd;
8: ac := mbr; goto 0;

9: mar := ir; mbr := ac; wr;
10: wr; goto 0;

11: alu := tir; if n then goto 15;
12: mar := ir; rd;
13: rd;
14: ac := mbr + ac; goto 0;

15: mar := ir; rd;
16: ac := ac + 1; rd;
17: a := inv(mbr);
18: ac := ac + a; goto 0;

19: tir := lshift(tir); if n then goto 25;
20: alu := tir; if n then goto 23;

21: alu := ac; if n then goto 0;
22: pc := band(ir, amask); goto 0;

23: alu := ac; if z then goto 22;
24: goto 0;

25: alu := tir; if n then goto 27;
26: pc := band(ir, amask); goto 0;
27: ac := band(ir, amask); goto 0;

28: tir := lshift(ir + ir); if n then goto 40;
29: tir := lshift(tir); if n then goto 35;
30: alu := tir; if n then goto 33;

```

```

{ciclo principal}
{incrementa pc}
{salva y decodifica mbr}

{000x o 001x?}
{0000 o 0001?}

{0000 = LODD}

{0001 = STOD}

{0010 o 0011?}
{0010 = ADDD}

{0011 = SUBD}
{Nota: x - y = x + 1 + }

{010x o 011x?}
{0100 o 0101?}

{0100 = JPOS}
{realiza el salto}

{0101 = JZER}
{no se produce el salto}

{0110 o 0111?}
{0110 = JUMP}

{0111 = LOCO}

{10xx o 11xx?}
{100x o 101x?}
{1000 o 1001?}

```

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

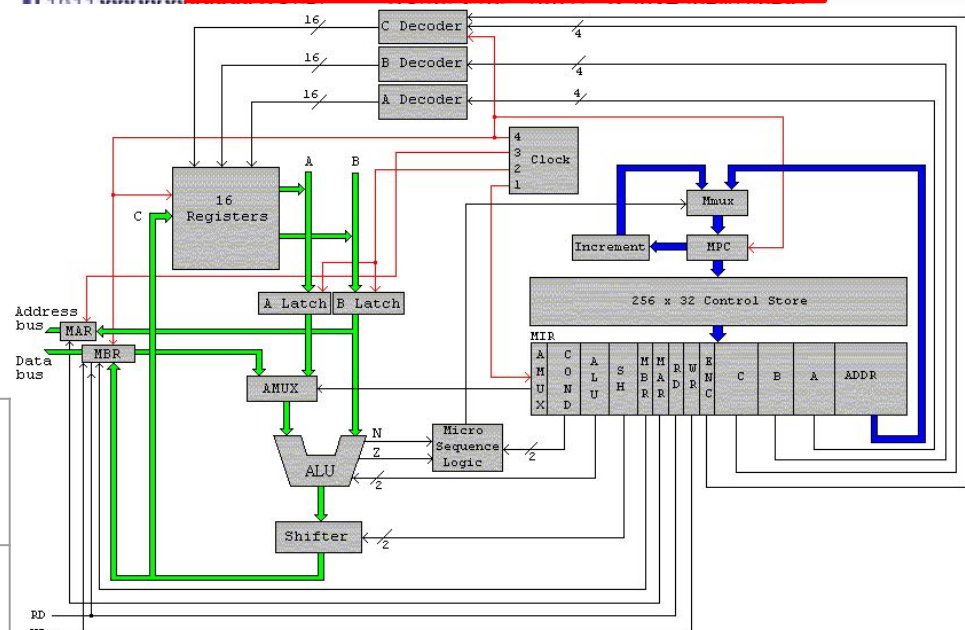
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8		3F8	0041						



0: $mar := pc; rd;$
 1: $pc := pc + 1; rd;$
 2: $ir := mbr; \text{if } n \text{ then goto 28;}$
 3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$
 4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$
 5: $alu := tir; \text{if } n \text{ then goto 9;}$
 6: $mar := ir; rd;$
 7: $rd;$
 8: $ac := mbr; \text{goto 0;}$
 9: $mar := ir; mbr := ac; wr;$
 10: $wr; \text{goto 0;}$
 11: $alu := tir; \text{if } n \text{ then goto 15;}$
 12: $mar := ir; rd;$
 13: $rd;$
 14: $ac := mbr + ac; \text{goto 0;}$
 15: $mar := ir; rd;$
 16: $ac := ac + 1; rd;$
 17: $a := inv(mbr);$
 18: $ac := ac + a; \text{goto 0;}$
 19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$
 20: $alu := tir; \text{if } n \text{ then goto 23;}$
 21: $alu := ac; \text{if } n \text{ then goto 0;}$
 22: $pc := band(ir, amask); \text{goto 0;}$
 23: $alu := ac; \text{if } z \text{ then goto 22;}$
 24: goto 0;
 25: $alu := tir; \text{if } n \text{ then goto 27;}$
 26: $pc := band(ir, amask); \text{goto 0;}$
 27: $ac := band(ir, amask); \text{goto 0;}$
 28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$
 29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$
 30: $alu := tir; \text{if } n \text{ then goto 33;}$

{ciclo principal}
 {incrementa pc}
 {salva y decodifica mbr}
 {000x o 001x?}
 {0000 o 0001?}
 {0000 = LODD}
 {0001 = STOD}
 {0010 o 0011?}
 {0010 = ADDD}
 {0011 = SUBD}
 {Nota: $x - y = x + 1 +$
 {010x o 011x?}
 {0100 o 0101?}
 {0100 = JPOS}
 {realiza el salto}
 {0101 = JZER}
 {no se produce el salto}
 {0110 o 0111?}
 {0110 = JUMP}
 {0111 = LOCO}
 {10xx o 11xx?}
 {100x o 101x?}
 {1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

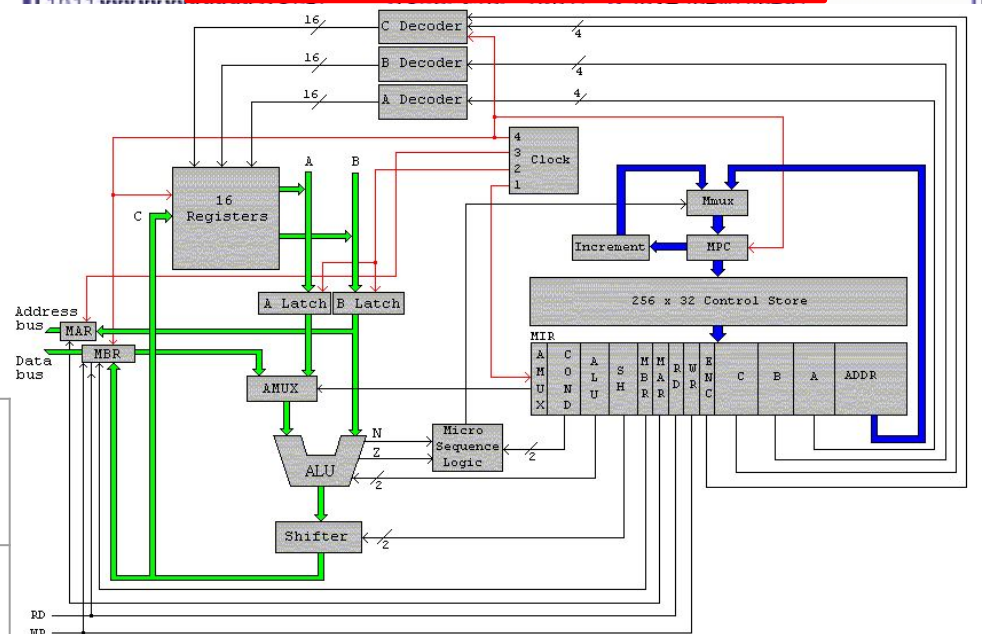
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	10 09	3F9	00 41						



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

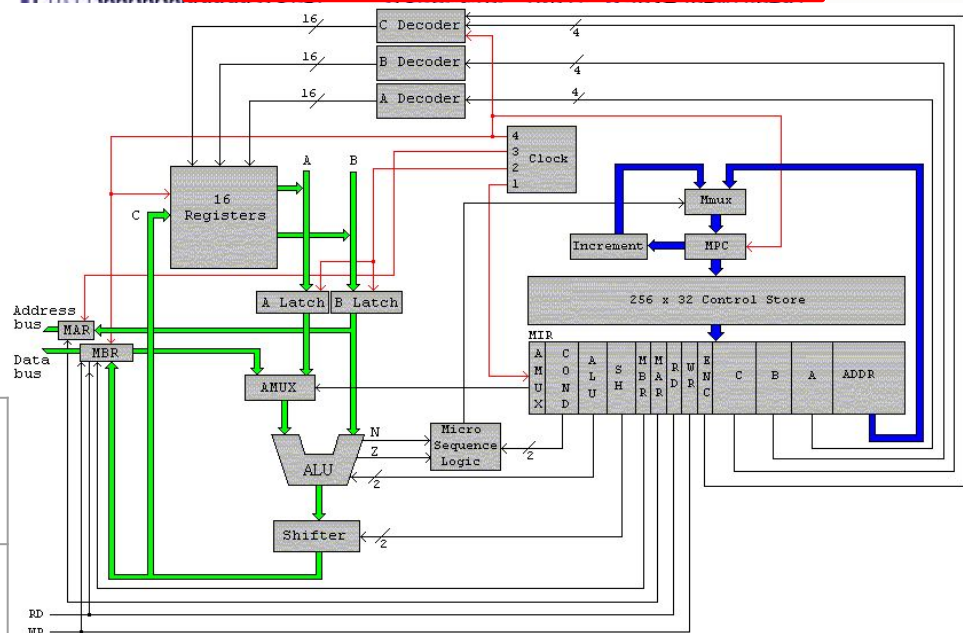
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	1009	3F9	0041	1009		1009	1009	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28};$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19};$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11};$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9};$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0};$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0};$	
11: $alu := tir; \text{if } n \text{ then goto 15};$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0};$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0};$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25};$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23};$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0};$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0};$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22};$	{0101 = JZER}
24: $\text{goto 0};$	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27};$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0};$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0};$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40};$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35};$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33};$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

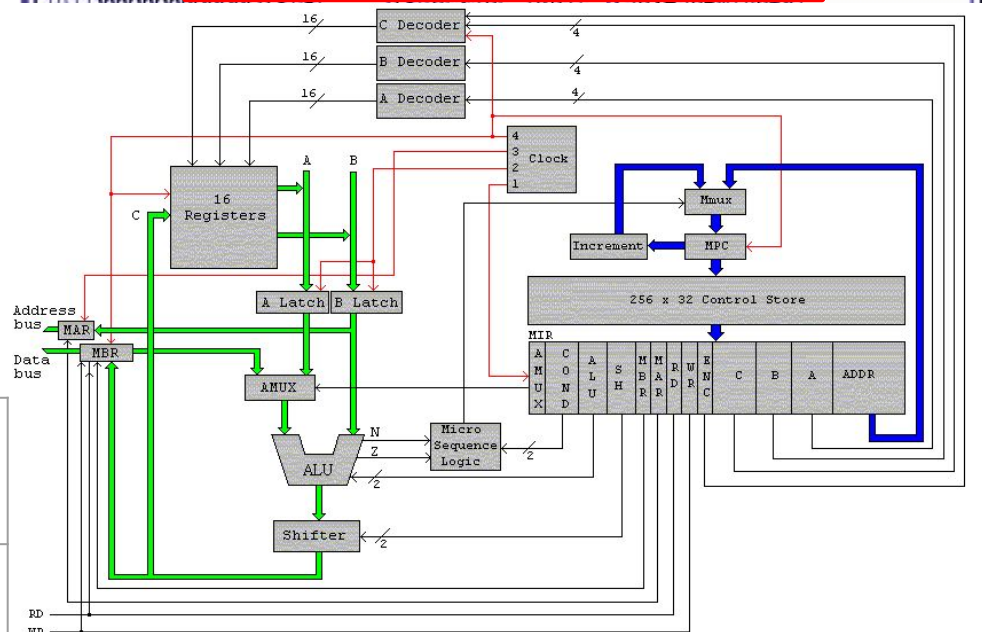
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	1009	3F9	0041	1009	4024	2012	4024	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{000x o 001x?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 o 0001?}
6: $mar := ir; rd;$	{0000 = LODD}
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

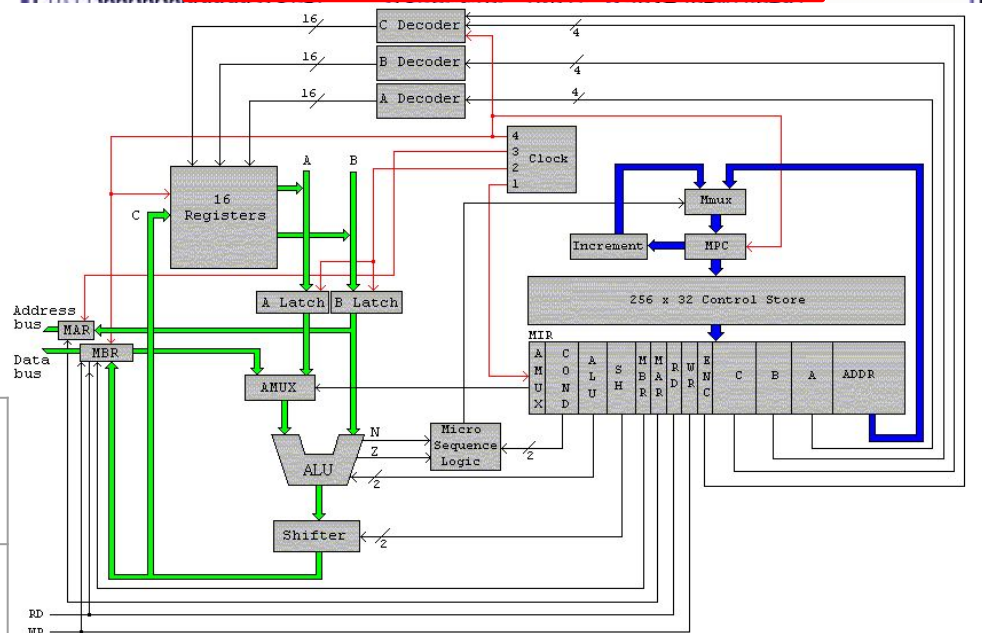
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	1009	3F9	0041	1009	8048	4024	8048	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{000x o 001x?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 o 0001?}
6: $mar := ir; rd;$	{0000 = LODD}
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

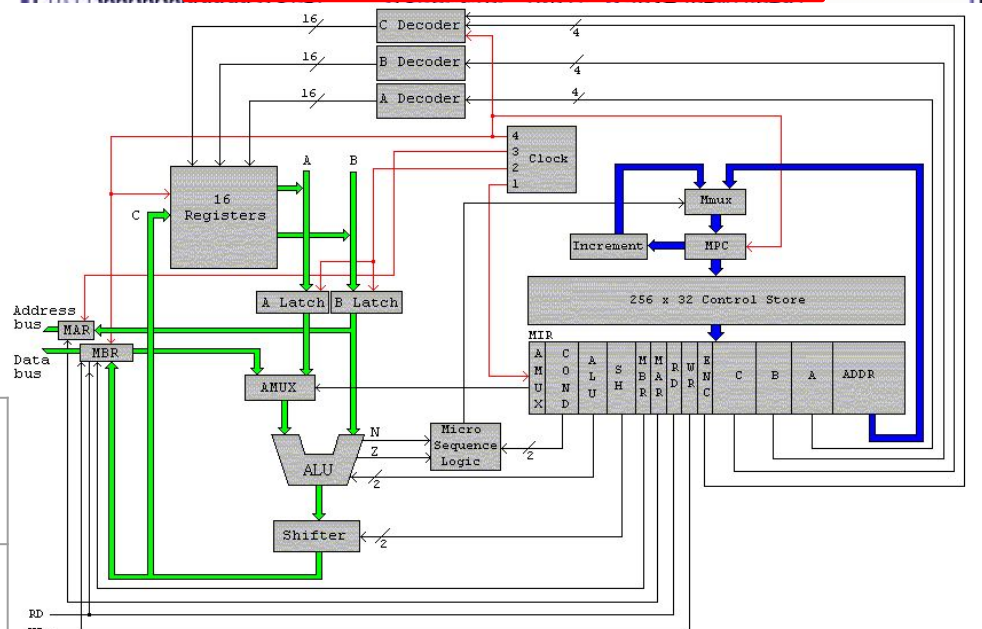
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F8	1009	3F9	0041	1009	8048	8048	8048	1	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

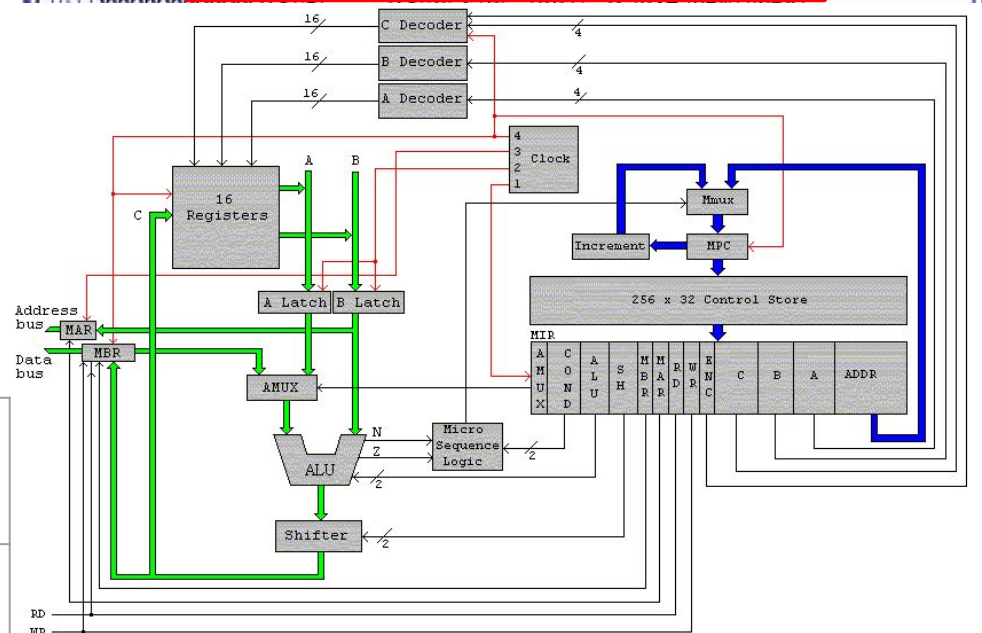
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- $mar:=ir$
- $mbr:=ac$
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
009	0041	3F9	0041	1009	8048	0041	0041	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

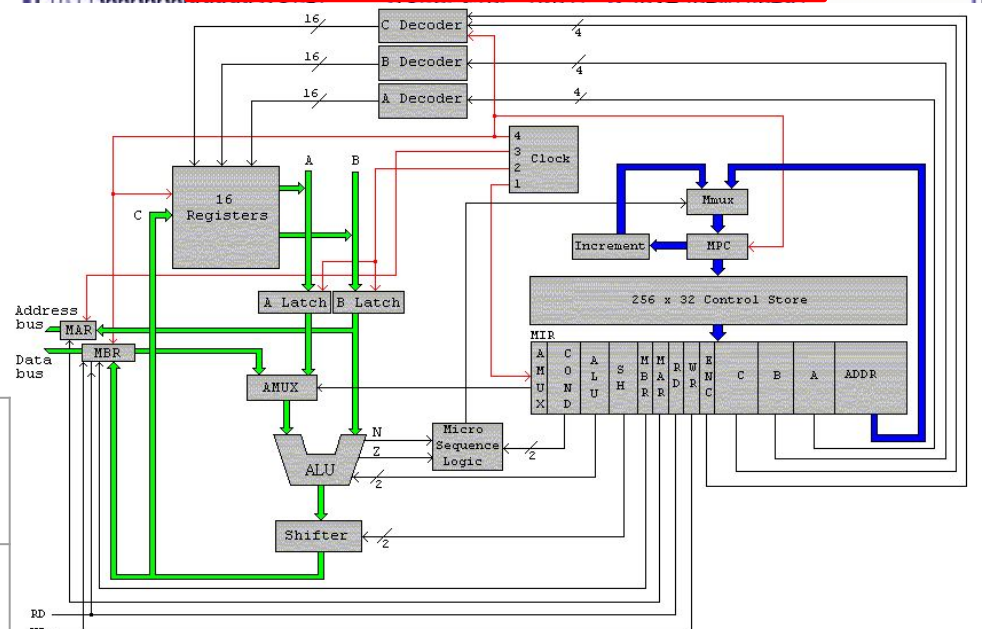
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- $mar:=ir$
- $mbr:=ac$
- wr

0001 0000 0000 1001

MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
009	0041	3F9	0041	1009	8048	0041	0041	0	



0: $mar := pc; rd;$

1: $pc := pc + 1; rd;$

2: $ir := mbr; \text{if } n \text{ then goto 28;}$

3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$

4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$

5: $alu := tir; \text{if } n \text{ then goto 9;}$

6: $mar := ir; rd;$

7: $rd;$

8: $ac := mbr; \text{goto 0;}$

9: $mar := ir; mbr := ac; wr;$

10: $wr; \text{goto 0;}$

11: $alu := tir; \text{if } n \text{ then goto 15;}$

12: $mar := ir; rd;$

13: $rd;$

14: $ac := mbr + ac; \text{goto 0;}$

15: $mar := ir; rd;$

16: $ac := ac + 1; rd;$

17: $a := inv(mbr);$

18: $ac := ac + a; \text{goto 0;}$

19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$

20: $alu := tir; \text{if } n \text{ then goto 23;}$

21: $alu := ac; \text{if } n \text{ then goto 0;}$

22: $pc := band(ir, amask); \text{goto 0;}$

23: $alu := ac; \text{if } z \text{ then goto 22;}$

24: goto 0;

25: $alu := tir; \text{if } n \text{ then goto 27;}$

26: $pc := band(ir, amask); \text{goto 0;}$

27: $ac := band(ir, amask); \text{goto 0;}$

28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$

29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$

30: $alu := tir; \text{if } n \text{ then goto 33;}$

{ciclo principal}

{incrementa pc}

{salva y decodifica mbr}

{000x o 001x?}

{0000 o 0001?}

{0000 = LODD}

{0001 = STOD}

{0010 o 0011?}

{0010 = ADDD}

{0011 = SUBD}

{Nota: $x - y = x + 1 +$

{010x o 011x?}

{0100 o 0101?}

{0100 = JPOS}

{realiza el salto}

{0101 = JZER}

{no se produce el salto}

{0110 o 0111?}

{0110 = JUMP}

{0111 = LOCO}

{10xx o 11xx?}

{100x o 101x?}

{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

STORE DIRECT

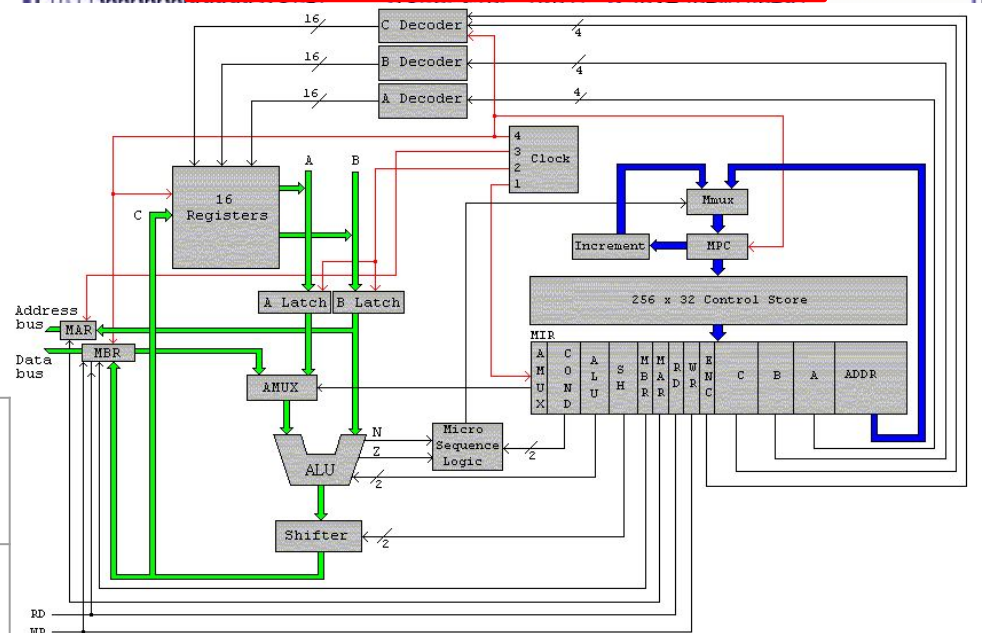
3F8 STOD 9

Guardar el valor del AC en la memoria(9)

- $mar := ir$
- $mbr := ac$
- wr

0001 0000 0000 1001

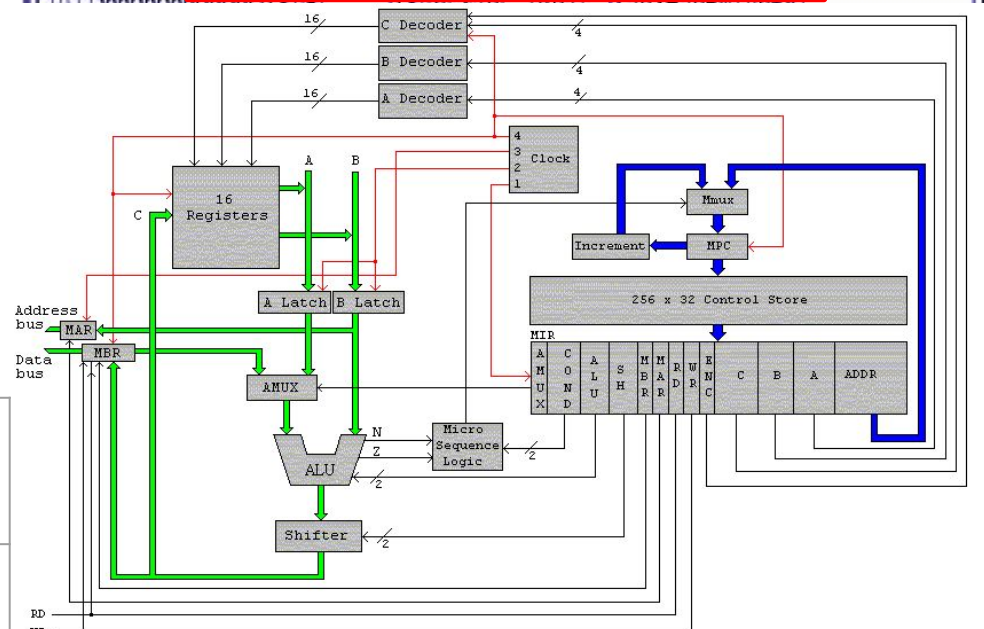
MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	0041	3F9	0041	1009	8048	0041	0041	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional • pc:=band(ir, amask) 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

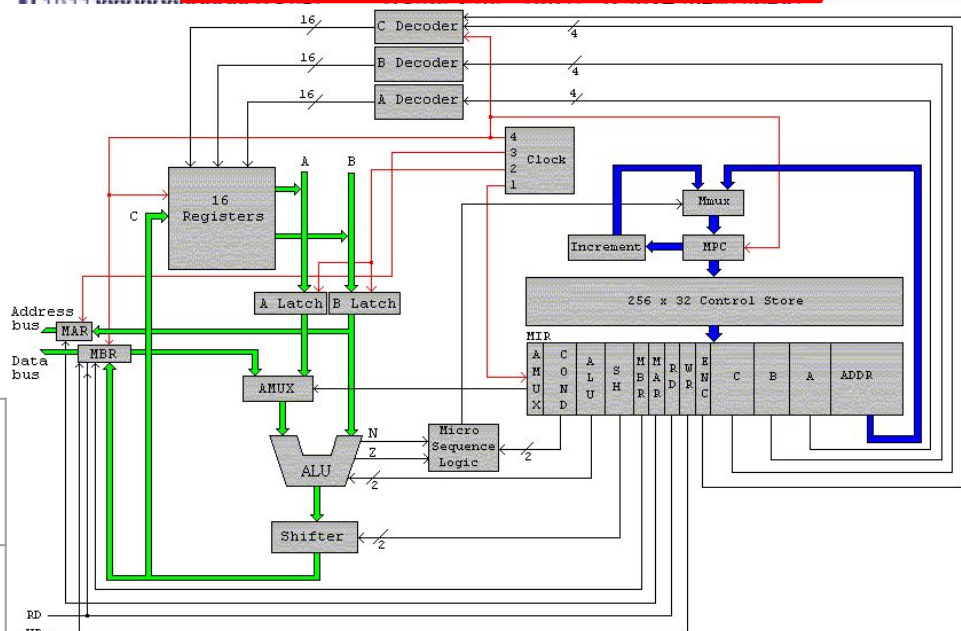
MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	10 09	80 48	00 41	00 41	0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

M	M	P	A	I	T	A	S	N	Z
A	B	C	C	R	I	L	H		
R	R				R	U	R		
3F9	63 F7	3FA	00 41	63 F7	80 48	63 F7	63 F7	0	

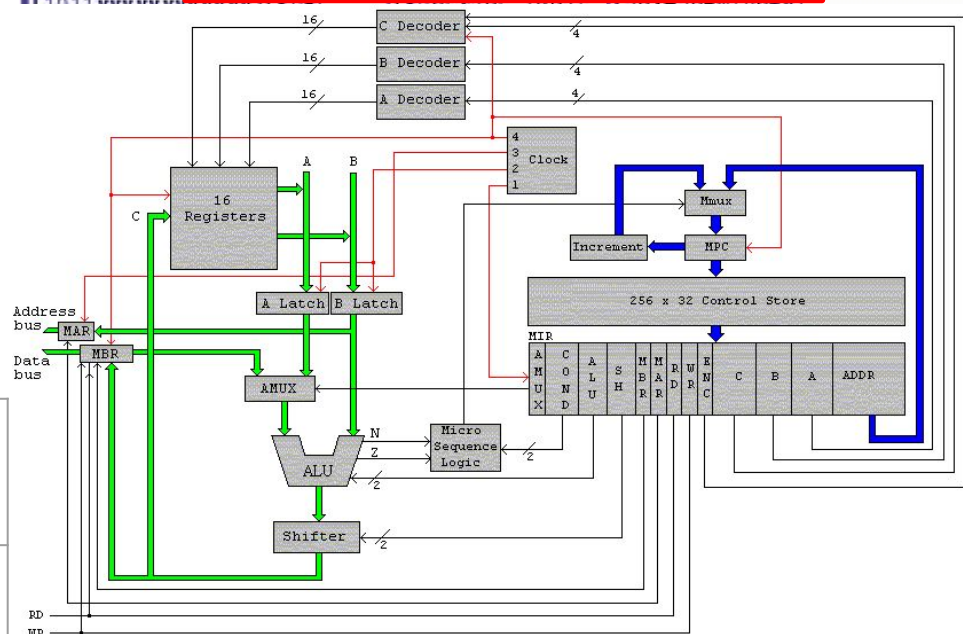
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional $pc := band(ir, amask)$ 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28};$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19};$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11};$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9};$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0};$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0};$	
11: $alu := tir; \text{if } n \text{ then goto 15};$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0};$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0};$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25};$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23};$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0};$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0};$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22};$	{0101 = JZER}
24: $\text{goto 0};$	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27};$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0};$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0};$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40};$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35};$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33};$	{1000 o 1001?}

M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F9	63 F7	3FA	00 41	63 F7	8F DC	C7 EE	8F DC	1	

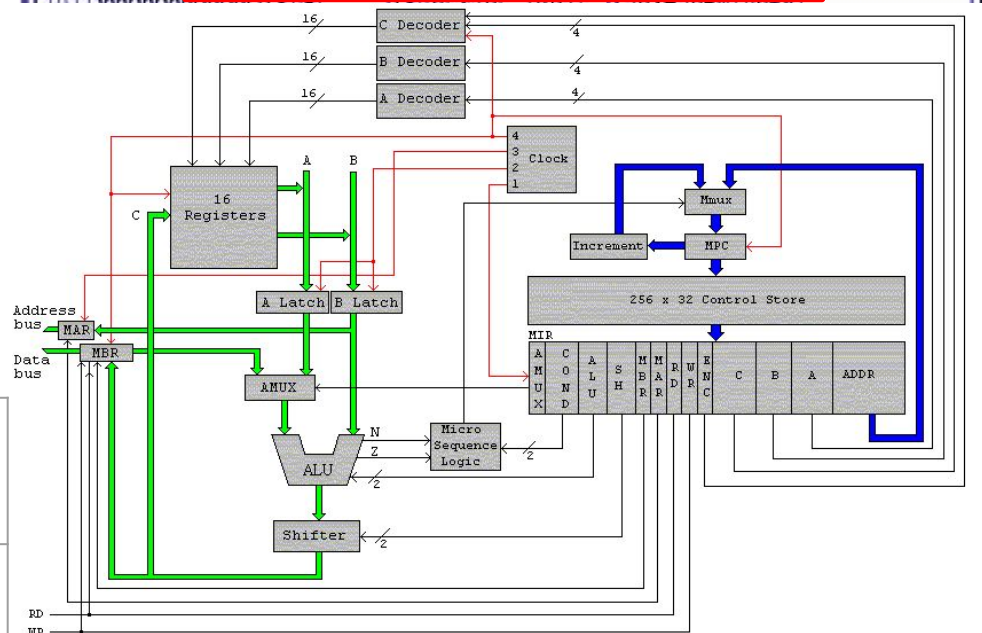
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional $pc := band(ir, amask)$ 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional • pc:=band(ir, amask) 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

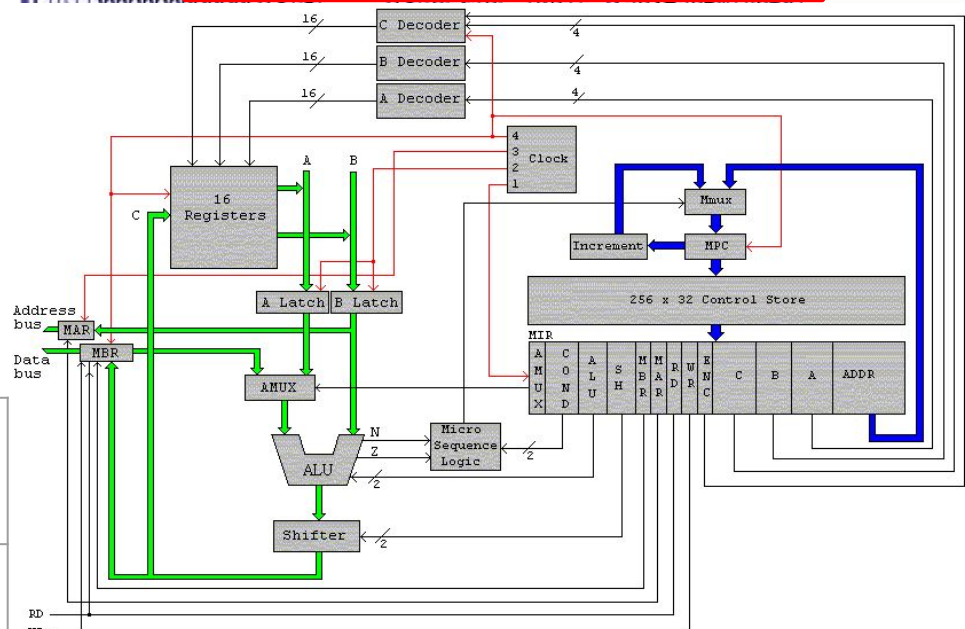
MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	8F DC	1F B8	1	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional • pc:=band(ir, amask) 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

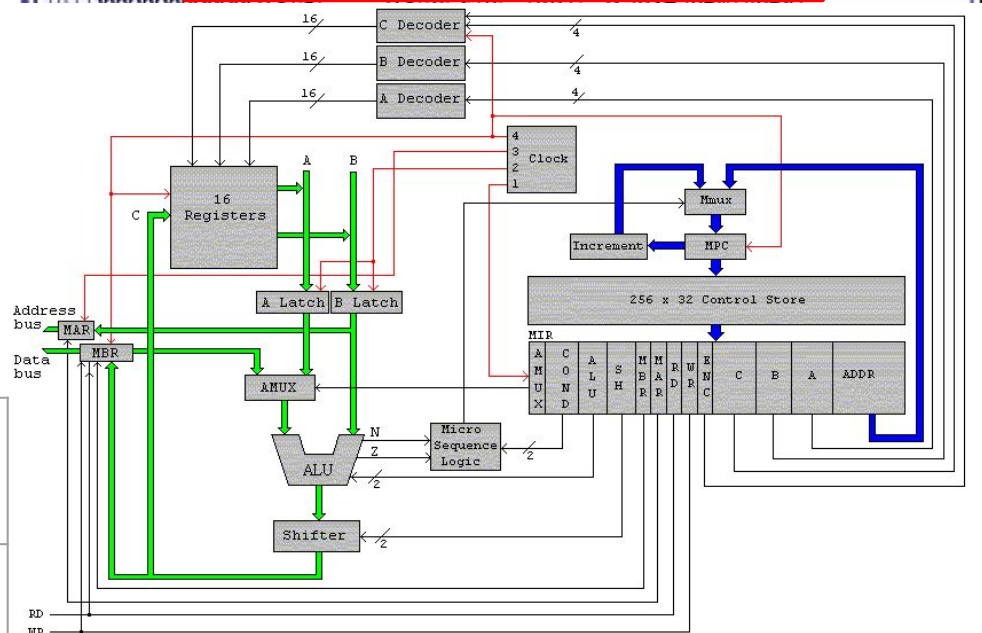
MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F9	63 F7	3FA	00 41	63 F7	1F B8	1F B8		0	



0: $mar := pc; rd;$	{ciclo principal}
1: $pc := pc + 1; rd;$	{incrementa pc}
2: $ir := mbr; \text{if } n \text{ then goto 28;}$	{salva y decodifica mbr}
3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$	{000x o 001x?}
4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$	{0000 o 0001?}
5: $alu := tir; \text{if } n \text{ then goto 9;}$	{0000 = LODD}
6: $mar := ir; rd;$	
7: $rd;$	
8: $ac := mbr; \text{goto 0;}$	
9: $mar := ir; mbr := ac; wr;$	{0001 = STOD}
10: $wr; \text{goto 0;}$	
11: $alu := tir; \text{if } n \text{ then goto 15;}$	{0010 o 0011?}
12: $mar := ir; rd;$	{0010 = ADDD}
13: $rd;$	
14: $ac := mbr + ac; \text{goto 0;}$	
15: $mar := ir; rd;$	{0011 = SUBD}
16: $ac := ac + 1; rd;$	{Nota: $x - y = x + 1 +$
17: $a := inv(mbr);$	
18: $ac := ac + a; \text{goto 0;}$	
19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$	{010x o 011x?}
20: $alu := tir; \text{if } n \text{ then goto 23;}$	{0100 o 0101?}
21: $alu := ac; \text{if } n \text{ then goto 0;}$	{0100 = JPOS}
22: $pc := band(ir, amask); \text{goto 0;}$	{realiza el salto}
23: $alu := ac; \text{if } z \text{ then goto 22;}$	{0101 = JZER}
24: goto 0;	{no se produce el salto}
25: $alu := tir; \text{if } n \text{ then goto 27;}$	{0110 o 0111?}
26: $pc := band(ir, amask); \text{goto 0;}$	{0110 = JUMP}
27: $ac := band(ir, amask); \text{goto 0;}$	{0111 = LOCO}
28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$	{10xx o 11xx?}
29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$	{100x o 101x?}
30: $alu := tir; \text{if } n \text{ then goto 33;}$	{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxx	JUMP 3F9 JUMP 3F7 Salto incondicional • pc:=band(ir, amask) 0110 0011 1111 0111		
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

M	M	P	A	I	T	A	S	N	Z
A	B	C	C	R	I	L	H		
R	R				R	U	R		
3F9	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	



0: $mar := pc; rd;$

1: $pc := pc + 1; rd;$

2: $ir := mbr; \text{if } n \text{ then goto 28;}$

3: $tir := lshift(ir + ir); \text{if } n \text{ then goto 19;}$

4: $tir := lshift(tir); \text{if } n \text{ then goto 11;}$

5: $alu := tir; \text{if } n \text{ then goto 9;}$

6: $mar := ir; rd;$

7: $rd;$

8: $ac := mbr; \text{goto 0;}$

9: $mar := ir; mbr := ac; wr;$

10: $wr; \text{goto 0;}$

11: $alu := tir; \text{if } n \text{ then goto 15;}$

12: $mar := ir; rd;$

13: $rd;$

14: $ac := mbr + ac; \text{goto 0;}$

15: $mar := ir; rd;$

16: $ac := ac + 1; rd;$

17: $a := inv(mbr);$

18: $ac := ac + a; \text{goto 0;}$

19: $tir := lshift(tir); \text{if } n \text{ then goto 25;}$

20: $alu := tir; \text{if } n \text{ then goto 23;}$

21: $alu := ac; \text{if } n \text{ then goto 0;}$

22: $pc := band(ir, amask); \text{goto 0;}$

23: $alu := ac; \text{if } z \text{ then goto 22;}$

24: goto 0;

25: $alu := tir; \text{if } n \text{ then goto 27;}$

26: $pc := band(ir, amask); \text{goto 0;}$

27: $ac := band(ir, amask); \text{goto 0;}$

28: $tir := lshift(ir + ir); \text{if } n \text{ then goto 40;}$

29: $tir := lshift(tir); \text{if } n \text{ then goto 35;}$

30: $alu := tir; \text{if } n \text{ then goto 33;}$

{ciclo principal}

{incrementa pc}

{salva y decodifica mbr}

{000x o 001x?}

{0000 o 0001?}

{0000 = LODD}

{0001 = STOD}

{0010 o 0011?}

{0010 = ADDD}

{0011 = SUBD}

{Nota: $x - y = x + 1 +$

{010x o 011x?}

{0100 o 0101?}

{0100 = JPOS}

{realiza el salto}

{0101 = JZER}

{no se produce el salto}

{0110 o 0111?}

{0110 = JUMP}

{0111 = LOCO}

{10xx o 11xx?}

{100x o 101x?}

{1000 o 1001?}

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxx	LODD	Load direct	$ac := m[x]$
0001xxxxxxx	STOD	Store direct	$m[x] := ac$
0010xxxxxxx			
0011xxxxxxx			
0100xxxxxxx			
0101xxxxxxx			
0110xxxxxxx			
0111xxxxxxx			
1000xxxxxxx			
1001xxxxxxx			
1010xxxxxxx			
1011xxxxxxx			

JUMP

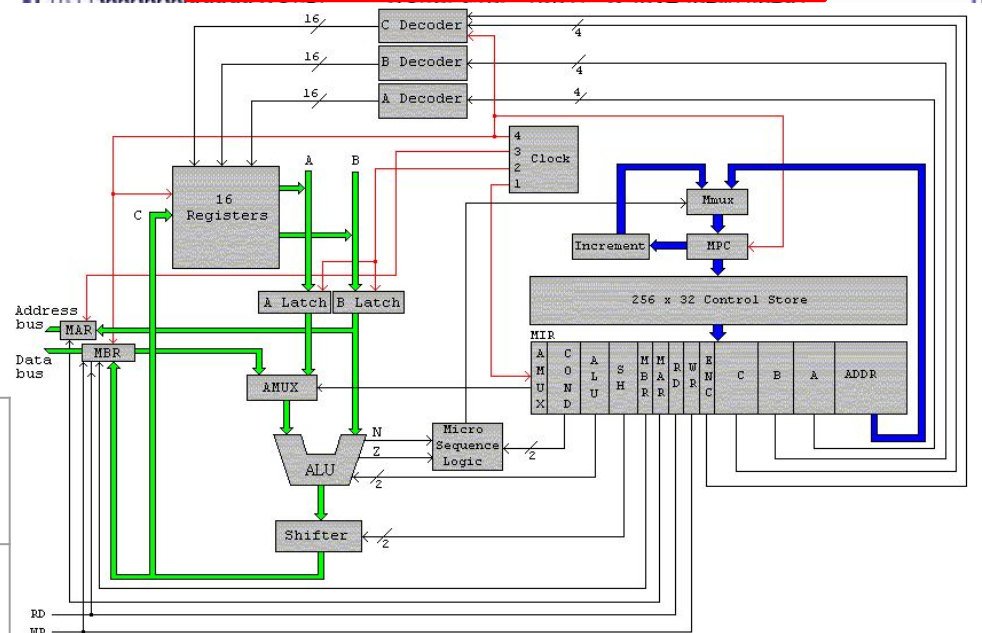
3F9 JUMP 3F7

Salto incondicional

- $pc := band(ir, amask)$

0110 0011 1111 0111

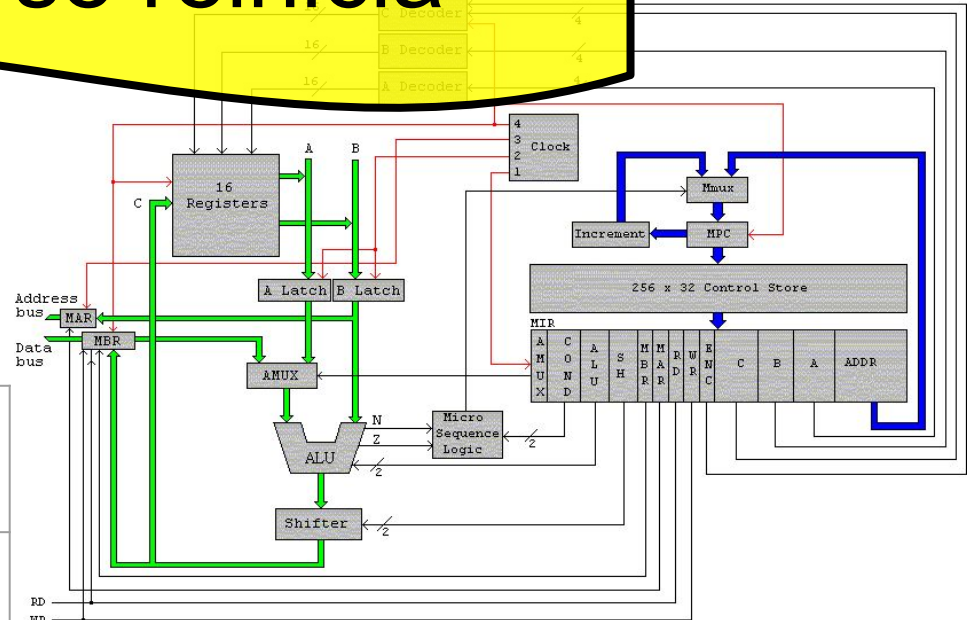
MAR	MBR	PC	AC	IR	TIR	ALU	SHR	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	



```
30: alu := tir ; if n then goto 33;
```

 $\{1000 \circ 1001?\}$

El ciclo se reinicia



M A R	M B R	P C	A C	I R	T I R	A L U	S H R	N	Z
3F7	63 F7	3F7	00 41	63 F7	1F B8	03 F7	03 F7	0	

Arquitectura Horizontal

T2 - Ejecución del micro programa

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta